Week – 1 Introduction to Data Structures

Introduction to Data Structures

Data items are represented within a computer as a sequence of binary digits. These sequences can appear very similar but have different meanings since computers can store and manipulate different types of data. For example, the binary sequence 01001100110010110101110011011100 could be a string of characters, an integer value, or a real value. To distinguish between the different types of data, the term **type** is used. Itrefers to a collection of values.The term **data type**refers to a given type along with a collection of operations for manipulating values of the given type. Programming languages commonly provide data types as part of the language itself.

A data structure is a specific way to store and organize data in a computer's memory.

Classification of Data Structure

Data structure is classified into 2 categories –

1. Primitive data structure
2. Non-Primitive data structure

The **primitive or simple data types** consist of values in the most basic form and cannot be decomposed into smaller parts. It consists of single values.

Python has four primitive data types- Integers, Float, Strings, Boolean

Integers - An integer represent numeric data, i.e., positive or negative whole numbers. Ex: 4, 5, or -1.

Float - Float stands for 'floating point number'. It can be rational numbers, usually ending with a decimal point, such as 1.11 or 3.14

String - Strings are collections of alphabets, words or other characters. In Python, strings are created by enclosing a sequence of characters within a pair of single or double quotes. For example: 'cake', "cookie", etc.

Boolean – It can take up the values: True and False, or integers 1 and 0. Booleans are useful in conditional and comparison expressions.
The primitive types provided by a language may not be sufficient for solving large complex problems. Thus, most languages allow for the construction of additional data types.

The **Non-primitive data** structure is a data structure that contains more than one data item**,** either of the **sa**me type or different types.

Nonprimitive data structures are further categorized into –

- Built-in data structure
- User defined data structures.

Built-in data structures include List, Tuple, Set, strings, Dictionary and Files

Sets – A set is a collection of items which is *unordered*, *unchangeable* and *unindexed*. Sets are used to store multiple items in a single variable. Set contains one or more items, of the same type or different type, which are separated by comma and enclosed in { }. Set items are immutable.
Set items can be numbers (integer, float, complex), string and tuples. But list and dictionary objects are not accepted.
Example:
set1 = {"abc", 3.4, True, 40, "male"}

Tuple - A tuple is a collection of items which is ordered and immutable. A tuple contains one or more items, of the same type or different type, which are separated by comma and surrounded by parenthesis ( ).
Example
T1 = (10, 20, 45, “apple”)

List - List are used to store multiple items in a single variable. List is a collection of items of same or different type, separated by a comma and enclosed in the square bracket [ ]. List can have data items of any datatype i.e., integer, strings, float, Boolean (i.e., heterogenous). Lists are ordered, indexed and mutable.
E**xample:**
L1 = [5, 10, 15, "abc" , 20]

Dictionary - Dictionary is used to represent a group of key-values. It is an unordered collection of key-value pairs separated by commas enclosed within { }. Each key is separated from its value by a colon.

- **Keys** must be a single element. Keys are case-sensitive. Keys are immutable objects.
- **Value** can be any data type such as list, tuple, integer etc. values can be duplicated.

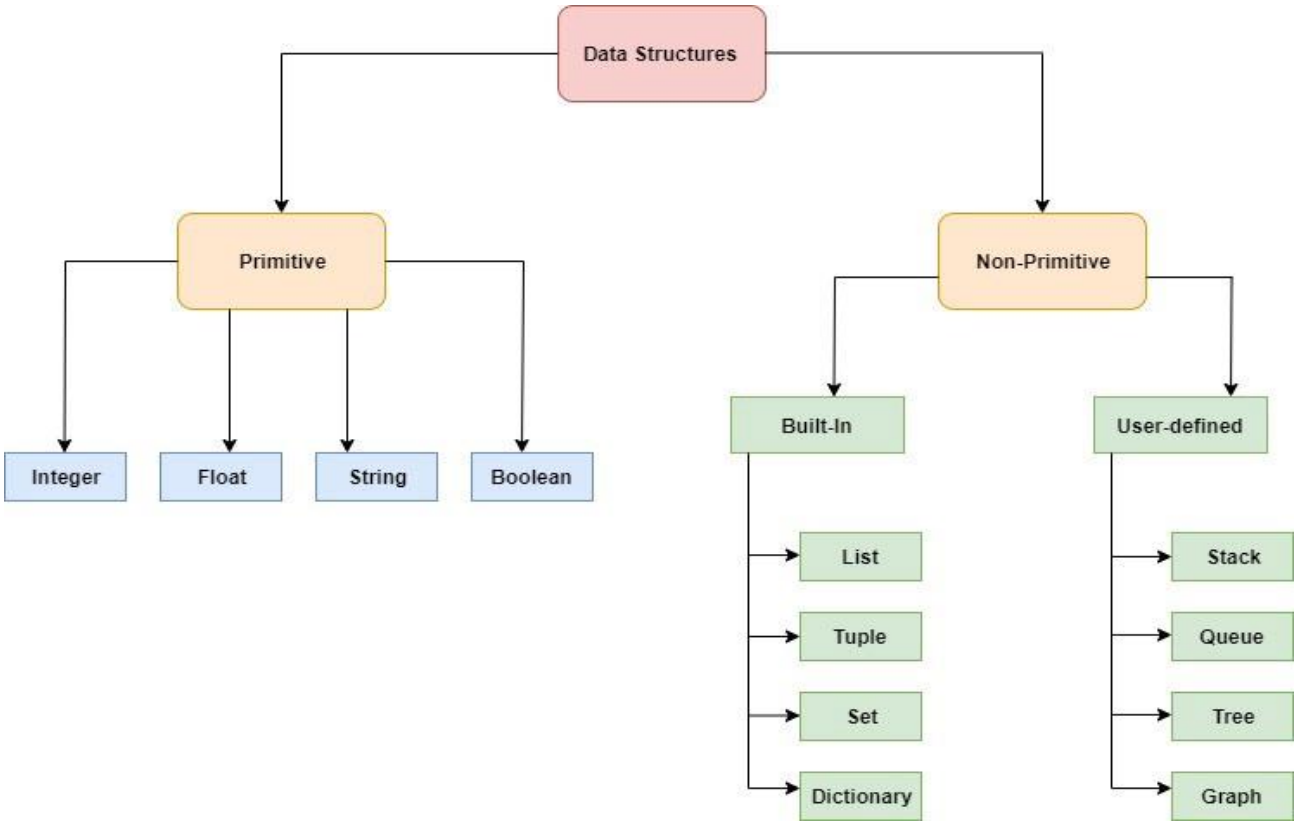
Users defined data structure are defined by the programmer and not the language.
Example: Stack, Tree, Queue, Graph etc.

Stacks - A stack is a container of objects that are inserted and removed according to the **Last-In-First-Out (LIFO)** concept. Adding elements to a stack, is known as a **push** operation, whereas removing or deleting an element is called a **pop** operation.

Queue - A queue is a container of objects that are inserted and removed according to the **First-In-First-Out (FIFO)** principle.

Tree: In tree, data has a hierarchical relationship among various elements.

Graph- A graph consist of nodes, also called vertices which may or may not be connected to each other. The **lines or the path** that connects two nodes is called an **edge**.



Characteristics of a Data Structure

- **Correctness** − Data structure implementation should implement its interface correctly.
- **Time Complexity** − Running time or the execution time of operations of data structure must be as small as possible.
- **Space Complexity** − Memory usage of a data structure operation should be as little as possible.

The data structure can also be classified as :

- Linear Data Structure
- Non-linear Data Structure

Linear Data Structure

A data structure is said to be linear if its elements combine to form any specific order.

Examples of the linear data structure are:

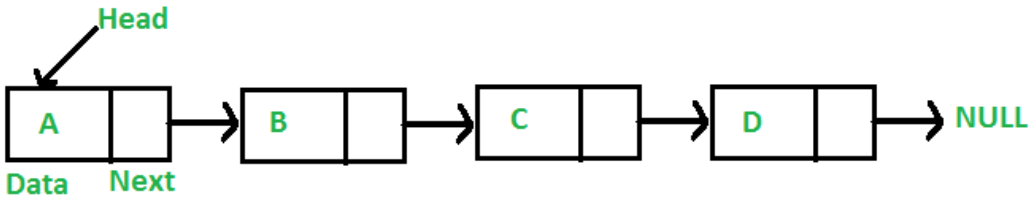
- Arrays
- Queues
- Stacks
- Linked lists

Array - An array is defined as a collection of items of same type stored in contiguous memory location. It is used to store multiple values in single variable. Array holds fixed number of elements.

Stacks - A stack is a container of objects that are inserted and removed according to the **Last-In-First-Out (LIFO)** concept. Stacks can be implemented using lists in Python. Adding elements to a stack, is known as a **push** operation, whereas removing or deleting an element is called a **pop** operation.

Queue - A queue is a container of objects that are inserted and removed according to the **First-In-First-Out (FIFO)** principle.

Linked list - A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers as shown in the below image:



Each node in a list consists of two parts:

- Data
- Pointer (Or Reference) to the next node

A linked list is represented by a pointer to the first node of the linked list. The first node is called the head. If the linked list is empty, then the value of the head is NULL.

Nonlinear Data Structure

This structure is used for representing data that contains a hierarchical relationship among various elements.

Examples of Non-Linear Data Structures are :

- Graphs
- Trees

Tree: In tree, data has a hierarchical relationship among various elements.

Graph- A graph consist of nodes, also called vertices which may or may not be connected to each other. The **lines or the path** that connects two nodes is called an **edge**. If the edge has a particular direction of flow, then it is a **directed graph**, with the direction edge being called an **arc**. If, no direction of flow on edge are specified, the graph is called an **undirected graph**.

Operations on data structures

The common operations that can be performed on the data structures are:

- **Searching:** Search for any element in a data structure.
- **Sorting:** Sort the elements of a data structure either in an ascending or descending order.
- **Insertion:** Insert the new element in a data structure.
- **Updation:** Update the element, i.e., can replace the element with another element.
- **Deletion:** The delete operation to remove the element from the data structure.

Example:Python program to use and demonstrate basic data structures

```
def setoperation():
    s1 = {1, 2, 5, 4}
    s2 = {7, 9, 2, 1}
    s1.add(6)
    print("Set1 after adding 6 is  ", s1)
    s1.remove(4)
    print("Set1 after deleting 4 is ",s1 )
    print("Union of 2 sets = ",s1 | s2)
    print("Intersection of 2 sets = ", s1 & s2)
    return

def tupleoperation():
    t1 = (11, 44, 55, 99)
    t2 = (22, 11, 66)
    print("Tuple after repetating elements for 2 times", t1*2)
    print("After concatenating 2 tuples = ", t1 + t2)
    print("Element at position 3 in tuple1 = ",t1[3])
    print("Elements from 1 to 3 in tuple1 =",t1[1:4])
    return

def listoperation():
    L1 = [10, 20, 30, 40]
    L1.append(15)
    print("list after adding 15 is ", L1)
    L1.insert(3,25)
    print("List after inserting 25 at position 3 is ", L1)
    print("Element at position 0 is ", L1[0])
    print("Elements in list from 1 to 3 is ",L1[1:4])
    print
    return

def dictoperation():
    d1 = {1:"One", 2:"Two", 3:"Three", 4:"Four"}

    print("Dictionary value for key 2 is ", d1[2])
    print("Dictionary items =",d1.items())
    print("Keys of dictionary d1 =", d1.keys())
    print("Values of dictionary d1 =", d1.values())
    return
```

```
# Main
while(True):
    print("    Menu ")
    print("----------------")
    print("1: Set Operation")
    print("2: Tuple Operation")
    print("3: List Operation")
    print("4: Dicitionary Operation")
    print("5: Exit")

    option = int(input("Enter option value  1 to 5 :  "))
    if option == 1:
        setoperation()
    elif option == 2:
        tupleoperation()
    elif option == 3:
        listoperation()
    elif option == 4:
        dictoperation()
    else:
        print("Invalid option value")
        break
```

Output:

```
    Menu
----------------
1: Set Operation
2: Tuple Operation
3: List Operation
4: Dicitionary Operation
5: Exit
Enter option value  1 to 5 :  2
Tuple after repetating elements for 2 times (11, 44, 55, 99, 11, 44, 55, 99)
After concatenating 2 tuples =  (11, 44, 55, 99, 22, 11, 66)
Element at position 3 in tuple1 =  99
Elements from 1 to 3 in tuple1 = (44, 55, 99)
    Menu
----------------
1: Set Operation
2: Tuple Operation
3: List Operation
4: Dicitionary Operation
5: Exit
Enter option value  1 to 5 :  5
Invalid option value
```

Abstraction

Abstraction is the process of hiding implementation details and showing only the essential features to the user.

It tells what an object does, not how it does it.

Abstraction is needed because,

- Reduces complexity
- Improves security
- Makes programs easy to understand
- Helps in modular programming

Real-Life Example: **ATM Machine**

- What we see: withdraw, deposit, check balance
- What is hidden: internal processing, database operations

Abstract Data Types

An abstract data type *(or ADT) is a programmer-defined data type that specifies a set of data values and a collection of well-defined operations that can be performed on those values.*

Abstract data types are defined independent of their implementation, allowing to focus on the use of the data type instead of how it's implemented. This is known as **information hiding.**

Abstract data types can be viewed like black boxes as shown in Figure. User programs interact with instances of the ADT by invoking one of the several operations defined by its interface.

The set of operations can be grouped into four categories:

- **Constructors:** Create and initialize new instances of the ADT.
- **Accessors:** Return data contained in an instance without modifying it.
- **Mutators:** Modify the contents of an ADT instance.
- **Iterators:** Process individual data components sequentially.

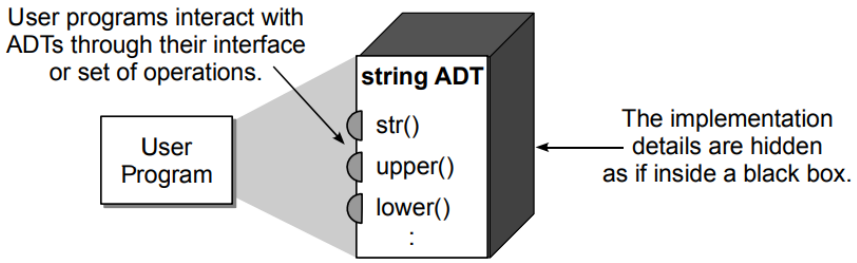


Figure shows Separating the ADT definition from its implementation.

The implementation of the various operations is hidden inside the black box, the contents of which need not to know in order to utilize the ADT.

Advantages of ADT

- ADT focus on solving the problem instead of implementation details. Makes programs easy to understand
- Programs can be divided into independent modules. So easy to manage large programs. Changes in implementation do not affect the user. So Easy to modify or improve code and reduces errors.
- Same ADT can be reused in different programs. Saves time and effort and Promotes code consistency
- ADT is reusable, robust, and is based on principles of Object-Oriented Programming (OOP) and Software Engineering (SE).

Example: **Defining the ADT**

Python allows classes to define. Define all ADT operations as named methods. This approach allows us to focus on the general ADT specification that can be easily translated to other languages.

Student ADT

GetDetails( ) : Get input from user on id, name, branch, m1, m2, m3.

display( ) :displaysthe id, name, branch details of student.

result( ) : get user input for marks, if the marks is equal to or greater than 40, then “pass”, otherwise “Fail”.

Searchid( ) : Get user input for *ID* to be searched, if it matches the data in the list, print the student data otherwise print "Student does not exist".

SearchName( ) : Get user input for Name to be searched, if it matches the data in the list, print the student data otherwise print "Student does not exist".

disp( ) : displaysthe id, name, branch details of student in tabular format.

Using the ADT

Abstraction focuses on what functionality the ADT provides instead of how that functionality is implemented. By hiding the implementation details, ADT can be used independent of its implementation.

```
#Student class
class Student():
    #Details method to get the student marks and name
    def GetDetails(self):
        print("Enter student Details : ")
        self.id = input("Id : ")
        self.name = input("Name : ")
        self.branch = input("Branch : ")
        self.m1 = int(input("Enter subject1 marks :"))
        self.m2 = int(input("Enter subject2 marks :"))
        self.m3 = int(input("Enter subject3 marks :"))
        return

    def display(self):
        print("{:10} {:10}".format('ID :',self.id))
        print("{:10} {:10}".format('NAME :',self.name))
        print("{:10} {:10}".format('BRANCH :',self.branch))
        return
```

```
    #result method to check whether student is pass or fail
    def result(self):
        self.display()
        self.pm = 40
        if(self.m1 >= self.pm):
            print("M1 :      ",self.m1,"Pass")
        else:
            print("M1 :      ",self.m1,"Fail")

        if(self.m2 >= self.pm):
            print("M2 :      ",self.m2,"Pass")
        else:
            print("M2 :     ",self.m2,"Pass")

        if(self.m3 >= self.pm):
            print("M3 :      ", self.m3,"Pass")
        else:
            print("M3 :      ",self.m3, "Fail")
        return
```

```
    def Searchid(self, id):
        if id == self.id:
            return True
        else:
            return False

    def SearchName(self, name):
        if name == self.name:
            return True
        else:
            return False

    def disp(self):
        print("{:10} {:10} {:10}".format(self.id,self.name, self.branch))
        return
```

```
# Main
n = int(input("Enter Total No. of students?"))
L = []
for i in range(n):
    s= Student()
    s.GetDetails()
    L.append(s)

# Display student details
for i in L:
    i.display()
```

```
#Display student all student details in tabular format
print("{:10} {:10} {:10}".format('ID :','NAME', 'BRANCH'))
print("=====================================")
for i in L:
    i.disp()

#Search by name
id = input("Enter Id of student U Want to Search?")

found = False
for e in L:
    found = e.Searchid(id)

    if (found):
        e.result()
        break
if (not found):
    print("Student Not Exist")
```

```
#Search by name
name = input("Enter Name of student U Want to Search?")

found = False
for e in L:
    found = e.SearchName(name)

    if (found):
        e.result()
        break

if (not found):
    print("Student Not Exist")
```

Implementing ADT

After defining the ADT, we need to provide an implementation in an appropriate language. In the above example Python programming language is used.

~~~~~~*******~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~

Week – 2 Algorithm Analysis

Algorithm is a sequence of steps to solve a problem. It is designed to solve problems, but a given problem can have many different solutions.
The approach used to determine which solution is most efficient for a given problem is by measuring the execution time. The solution is implemented by constructing a computer program, using a given programming language. Then execute the program and time it.
The execution time is dependent on several factors -

1. The amount of data that must be processed directly affects the execution time. As the data set size increases, so does the execution time.
2. The execution times can vary depending on the type of hardware and the time of day a computer is used. If multi-process, multi-user system is used to execute the program, the execution of other programs on the same machine can directly affect the execution time of our program.
3. The choice of programming language and compiler used to implement an algorithm can also influence the execution time. Some compilers are better optimizers than others and some languages produce better optimized code than others. Thus, we need a method to analyse an algorithm's efficiency independent of the implementation details.

There are two techniques to measure the efficiency of an algorithm.

- **Time Complexity:** Time complexity is the time taken by an algorithm to execute.
- **Space Complexity:** Space complexity is the amount of memory used by an algorithm while executing.

The analysis of algorithms is to find the complexity of algorithms in terms of time and storage

Space Complexity

Space complexity of an algorithm represents the amount of memory space required by the algorithm in its life cycle. The space required by an algorithm is equal to the sum of the following two components −

- A **fixed part** that is a space required to store certain data and variables, that are independent of the size of the problem. For example, simple variables and constants used, program size, etc.
- A **variable part** is a space required by variables, whose size depends on the size of the problem. For example, dynamic memory allocation, recursion stack space, etc.

Space complexity S(P) of any algorithm P is S(P) = C + SP(I), where C is the fixed part and S(I) is the variable part of the algorithm, which depends on instance characteristic I. Following is a simple example to explain the concept −

Algorithm: SUM(A, B)

Step 1 − START

Step 2 − C ← A + B + 10

Step 3 − Stop

Here we have three variables A, B, and C and one constant. Hence S(P) = 1 + 3. Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

Asymptotic Notations

Instead of counting the precise number of operations or steps, computer scientists are more interested in classifying an algorithm based on the order of magnitude as applied to execution time or space requirements. This classification approximates the actual number of required steps for execution or the actual storage requirements in terms of variable-sized data sets.

Asymptotic Notation is used to describe the running time of an algorithm - how much time an algorithm takes with a given input, n.

The commonly used asymptotic notations to calculate the running time complexity of an algorithm.

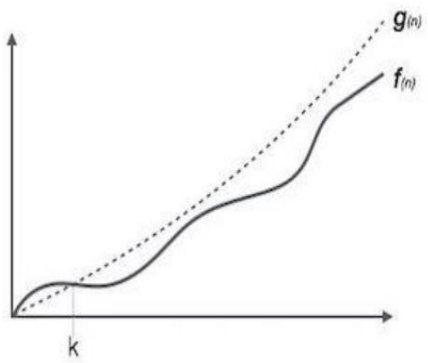
- O Notation
- Ω Notation
- θ Notation

Big O Notation

The Big O notation, O(n) is the formal way to express the upper bound of an algorithm's running time. It measures the worst-case time complexity or the longest amount of time an algorithm can possibly take to complete.

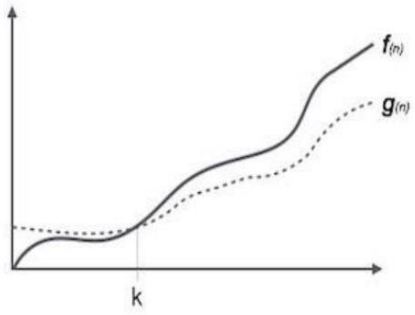
Example: If there is a linear relationship between the input and the step taken by the algorithm to complete its execution, the Big-O notation used will be O(n).

$$f(n) = O(g(n))$$



Big Ω Notation

The notation Ω(n) is the formal way to express the lower bound of an algorithm's running time. It measures the best-case time complexity or the best amount of time an algorithm can possibly take to complete.

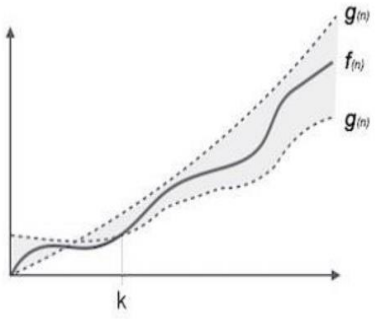


$$f(n) = \Omega(g(n))$$

Big θ Notation

The notation θ(n) is the formal way to express both the lower bound and the upper bound of an algorithm's running time.

Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.It is represented as follows.



$$f(n) = \Theta(g(n))$$

Common Big O Notations

Thefollowing are some of the **most common important Big O Notation functions**

$f(\cdot)$	Common Name
1	constant
$\log n$	logarithmic
n	linear
$n \log n$	log linear
n^2	quadratic
n^3	cubic
a^n	exponential

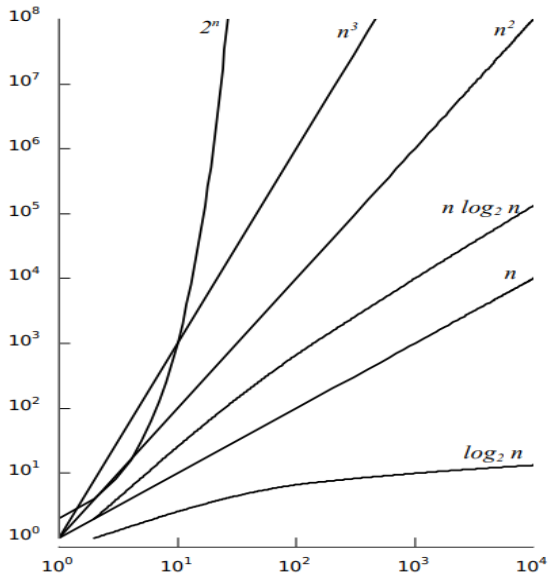
A **logarithmic** algorithm has time-complexity O(**$\log_a$** n). These algorithms are generally **very efficient since $\log_a n$ will increase more slowly than n.**

(a will typically equal 2 and thus the notation log n means $\log_2$ n).

Polynomial algorithms with an efficiency expressed as a polynomial of the form

$$a_m n^m + a_{m-1} n^{m-1} + \ldots + a_2 n^2 + a_1 n + a_0$$

are characterized by a time-complexity of $O(n^m)$. The most common polynomial algorithms are **linear** (m=1), **quadratic** (m=2), and **cubic** (m= 3).



Input Size →

An algorithm whose efficiency is characterized by a term in the form a^n is called **exponential**. **Exponential algorithms are among the worst algorithms in terms of time-complexity**.

When evaluating the time complexity of an algorithm or code segment, we assume that basic operations only require constant time. The basic operations include statements and function calls whose execution time does not depend on the specific values of the data that is used or manipulated by the given instruction. For example, the assignment statement, evaluation of arithmetic and logical expressions, subscript operator when used with Python's sequence types (strings, tuples, and lists) is also a basic instruction

Constant Time Complexity – O(1)

In **constant time complexity,** the execution time of an algorithm doesn't change and remains constant irrespective of the size of the input data.

Consider a list **list1 = [4, 6, 7, 1, 5, 2]**.

Accessing a specific element using indexing takes a constant amount of time.

Example: python code to demonstrate constant time complexity

```
def listelement(e):
    print("First Element",e[0])
    print("Last Element",e[-1])
    print("======================")
    return

listelement([1,2])
listelement([1,2,3,4])
listelement([2, 3, 4, 1, 6, 7, 8, 9, 12, 10])

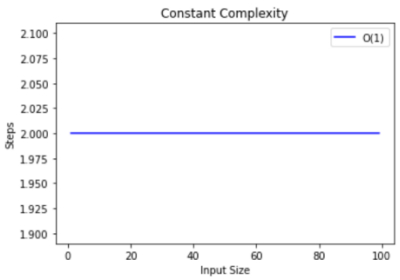
# To plot graph
import matplotlib.pyplot as p

p.xlabel('Input Size')
p.ylabel('Steps')
p.title('Constant Complexity')
x = list( range(1,100))
y = [2 for y in x]

p.plot(x, y, 'b', label='O(1)')
p.legend()
p.show()
```

Output:

```
First Element 1
Last Element 2
======================
First Element 1
Last Element 4
======================
First Element 2
Last Element 10
======================
```



Linear Time Complexity – O(n)

In linear time complexity, the execution time of an algorithm/code increases linearly with the increase in the size of the input.

Example: the code below, adds all the elements of a list by traversing from its first element till its last element. Hence, the longer the input (list), theexecution time increases linearly with the increase in the size of the input.

```
def lineardemo(n):
    s = 0
    for i in n:
        s = s + i
    print("Sum = ",s)
    return

# Calling function
lineardemo([0])
lineardemo([0, 1, 2, 4])
lineardemo([i for i in range(100)])

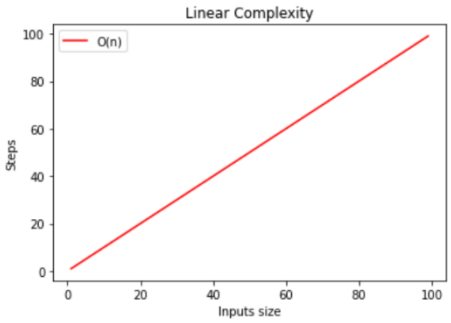
# Code to plot graph
import matplotlib.pyplot as p

p.title('Linear Complexity')
p.xlabel('Inputs size')
p.ylabel('Steps')

x = list(range(1,100))
y = [y for y in x]
p.plot(x, y, 'r', label = 'O(n)')
p.legend()
p.show()
```

Output:

```
Sum =  0
Sum =  7
Sum =  4950
```



Quadratic Time Complexity $O(n^2)$

In quadratic Time complexity, the execution time of an algorithm/code increases quadratically(square) with the increase in the size of the input.

Example:The outer loop iterates through each item, and then for each item in the outer loop, the inner loop iterates over each item. This makes total number of steps n*n where n is the input size.

```
def quadraticdemo(n):
    c = 0
    for i in range(n):
        for j in range(n):
            c = c + 1
    return c

print("Count = ",quadraticdemo(10))

# Graph for Quadratic Time Complexity
import matplotlib.pyplot as p

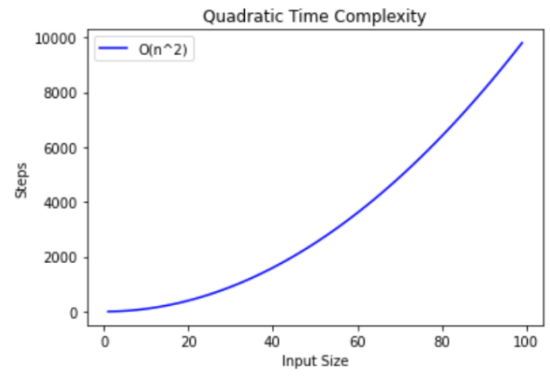
p.title('Quadratic Time Complexity')
p.xlabel('Input Size')
p.ylabel('Steps')

x = list(range(1,100))
y = [y*y for y in x]

p.plot(x, y, 'b', label = 'O(n^2)')
p.legend()
p.show()
```

Output:

```
Count =  100
```



Logarithmic Time Complexity

When the size of input is N but the number of steps to execute the code is log(N), such a code is said to be executing in Logarithmic Time complexity

An algorithm is said to have a logarithmic time complexity when it reduces the size of the input data in each step. Example: Binary search

Example: For each iteration of loop the value of " i " is reduced to half.

```
def logarithmicdemo(n):
    c = 0
    i = n
    while i >=1 :
        c = c + 1
        i = i//2
    return c
print("Count = ",logarithmicdemo(16))

# Graph for logarithmic Time Complexity
import matplotlib.pyplot as p
import math

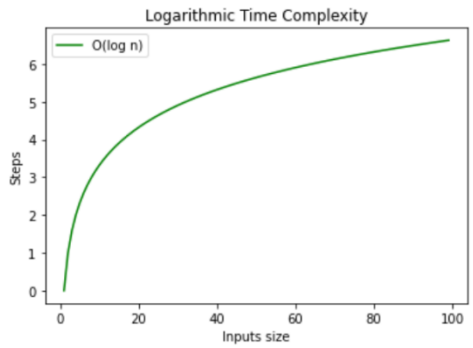
p.title('Logarithmic Time Complexity')
p.xlabel('Inputs size')
p.ylabel('Steps')

x = list(range(1,100))
y = [math.log(y,2) for y in x]

p.plot(x, y, 'g', label = 'O(log n)')
p.legend()
p.show()
```

Output:

```
Count =  5
```



Execution time or Runtime Analysis for Different input Cases – Best Case, Worst Case and Average Case

Some algorithms have different run times for different sets of inputs of the same size. i.e., an algorithm can perform differently based on the given input.

There are three cases to analyse an algorithm. They are

- **Worst case**
- **Average case**
- **Best case.**

Worst Case - In **worst case,** maximum time is required by an algorithm for the execution of a program.

Consider the following example, which traverses a list containing integer values to find the position of the first negative value. The input to this problem is the list consisting of n integer values.

```
def findnegative(L):
    for i in L:
        if i < 0:
            return L.index(i)
        return None
```

If the list does not contain a negative value,

L = [72, 4, 90, 56, 12, 67, 43, 17, 2, 86, 33]

p= findnegative(L)

the return statement inside the loop will not be executed and the loop will terminate after traversingall n times. In this case, the **function requires O(n) time**. This is known as the **Worst case** since the function must examine every value in the list requiring the greatest number of steps.

Best case - In **best case,** minimum time is required by an algorithm for the execution of a program.

Consider the case where the list contains a negative value in the first element:

L = [-12, 50, 4, 67, 39, 22, 43, 2, 17, 28]

p= findnegative(L)

There will be one iteration of the loop since the test of the condition by the if statement will be true the first time through and the return statement inside the loop will be executed. In this case, the**function requires O(1) time**. This is known as the best case since the function has to examine the first value in the list requiring the least number of steps.

Average Case - In average case, we take the average time of all the possible inputs. function the searches to iterate halfway through the list before finding the first negative value, which on average requires n/2 iterations.

The average case is more difficult to evaluate.

****

3. Algorithm design strategies

An ***algorithm design technique*** (or "strategy" or "paradigm") is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

Different algorithm design techniques are-

- **Brute Force**
- **Divide and Conquer Approach**
- **Greedy Technique:**
- **Dynamic Programming**
- **Branch and Bound**
- **Backtracking Algorithm**
- **Randomized Algorithm**

The **brute force approach** is a simple problem-solving method where **all possible solutions are tried one by one** until the correct solution is found.

The brute force method is ideal for solving small and simpler problems.

Example for brute force approach are -Bubble sort, Selection Sort, Linear Search.

Sorting is the process of arranging or ordering a collection of items such that each item and its successor satisfy a prescribed relationship. The items can be simple values, such as integers, real numbers, or more complex types, such as student records or dictionary entries.

Examples of sorting in everyday life are - the listings of a phone book, the words in a dictionary, all of which are organized in alphabetical order to make finding an entry much easier.

Use of sorting is –

- The efficiency of some applications can be improved when working with sorted lists.
- The presentation of data in some organized fashion.

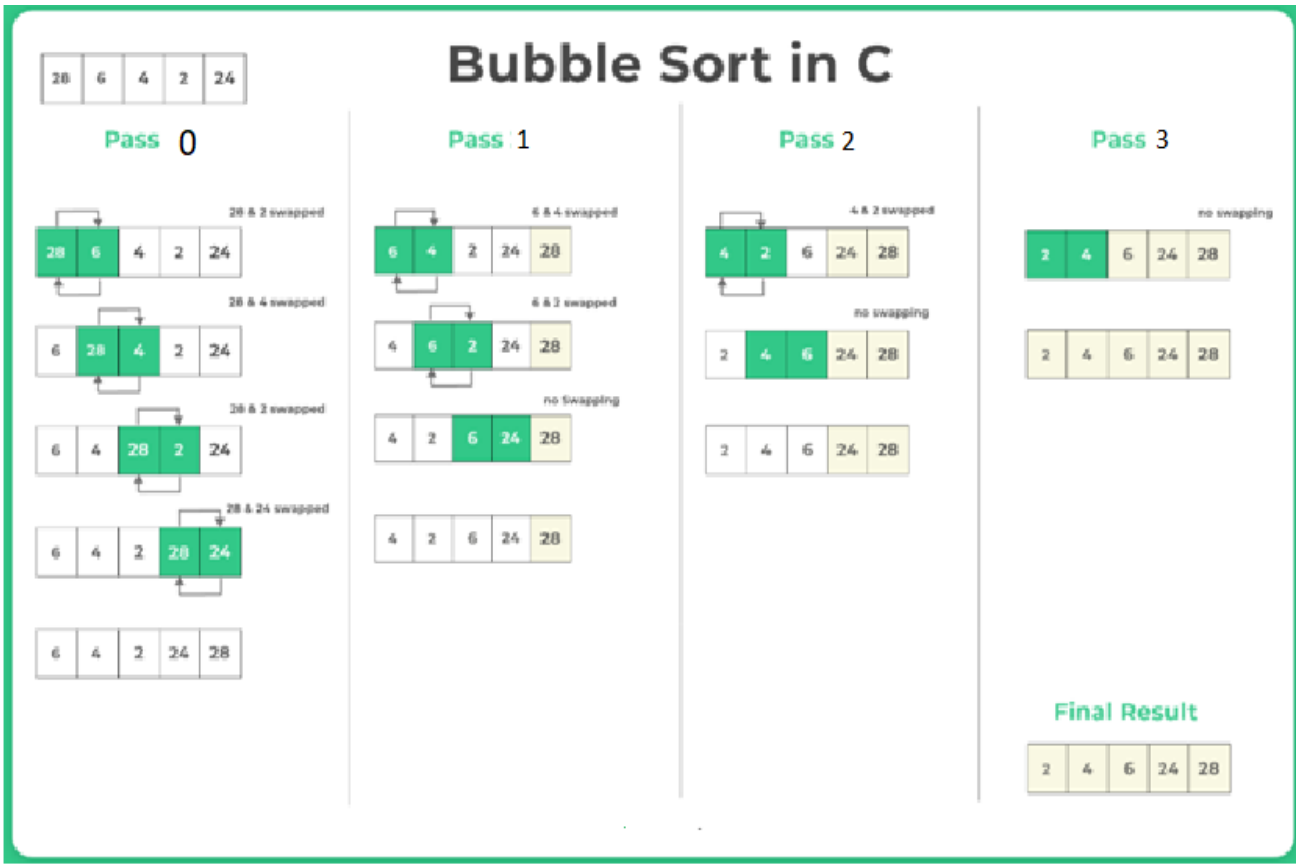
Bubble Sort

Bubble sort is a simple sorting algorithm. It rearranges the values by iterating over the list multiple times. In each iteration, compares each pair of adjacent elements and swaps them if they are in wrong order. This causes larger element to bubble to the end of the list.

Algorithm of bubble sort (for sorting a given list in ascending order):

1. Starting with the first element.
2. Compare the current element with the next element of the list.
3. If the current element is greater than the next element, then swap both the elements. If not, move the next element.
4. Repeat Step 1, 2, 3 for n-1 times to get sorted list.

Example:



```
def Bubblesort(List1):
    n =  len(List1)
    #
    for i in range(n-1):
        for j in range(n-i-1):
            # Compare adjacent elements
            if List1[j] > List1[j+1]:
                List1[j], List1[j+1] = List1[j+1], List1[j]

    return List1

# Main
List1 = [65, 4, 1, 8, 11, 23, 14, 9, 45]
print("Given List = ", List1)
slist = Bubblesort(List1)
print("Sorted List = ", slist)
```

Output:

```
Given List =  [65, 4, 1, 8, 11, 23, 14, 9, 45]
Sorted List =  [1, 4, 8, 9, 11, 14, 23, 45, 65]
```

Time Complexity Analysis of Bubble Sort

- **Best Case Complexity**: The bubble sort algorithm has a best-case time complexity of **O(n)** for the already **sorted list.**

  In this case, no swapping will happen in the first iteration. So, when this happens, break from the loop, after the very first iteration. Hence, time complexity in the best-case O(n) because it has to traverse through all the elements once.

- **Average Case Complexity**: The average-case time complexity for the bubble sort algorithm is **$O(n^2)$**, for **elements in random order**. i.e., neither in the ascending order nor in the descending order.

- **Worst Case Complexity**: The worst-case time complexity is**$O(n^2)$**, which occurs when we **sort the descending order of an array into the ascending order**. Given n elements, then in the first pass, it will do n-1 comparisons. In the second pass, it will do n-2, In the third pass, it will do n-3 and so on. Thus, the total number of comparisons is

```
Sum = (n-1) + (n-2) + (n-3) + ..... + 3 + 2 + 1
Sum = n(n-1)/2
```

Space Complexity

- Space complexity is O(1) because an extra variable is used for swapping.
- In the **optimized bubble sort algorithm**, two extra variables are used. Hence, the space complexity will be O(2).

Bubble Sort	
Time Complexity	
Best case	O(n)
Worst case	$O(n^2)$
Average case	$O(n^2)$
Space Complexity	O(1)

Optimized Bubble Sort

The bubble sort algorithm can be improved by having it terminate early and not require it to perform all n^2 iterations when the sequence is in sorted order. We can determine the sequence is in sorted order when no swaps are performed by the if statement within the inner loop. At that point, the function can return immediately without completing the remaining iterations

```
# Optimized Bubble sort
def Bubblesort(List1):
    n =  len(List1)
    swap = False
    for i in range(n-1):
        for j in range(n-i-1):
            if List1[j] > List1[j+1]:
                List1[j], List1[j+1] = List1[j+1], List1[j]
                swap = True
        if(swap == False):
            break
    return List1

List1 = [65, 4, 1, 8, 11, 23, 14, 9, 45]
Bubblesort(List1)
print("Sorted List = ", List1)
```

Output:

```
Sorted List =  [1, 4, 8, 9, 11, 14, 23, 45, 65]
```

Advantages of Bubble Sort

1. Easily understandable and code can be written easily for this algorithm.
2. No need of extra memory.
3. Minimal space requirement than that of other sorting algorithms.

Disadvantages of Bubble Sort

1. It does not work well when for large unsorted lists and takes more time.
2. It requires n^2 order of steps to sort an algorithm.

Selection Sort

The selection sort algorithm sorts list by repeatedly finding the minimum element from unsorted part and putting it in appropriate position.

The algorithm maintains two sub lists in a given list- sorted and unsorted.In every iteration of selection sort, the minimum element from the unsorted sub list is picked and moved to the sorted sub list.

Algorithm ofSelection sort

1. The outer **for** loop traverses the whole of the list. It initially sets the minimum to be the index of the first element in the unsorted list.

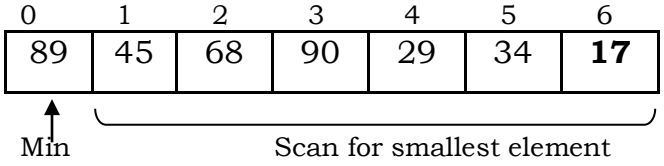
2. The inner **for** loop traverses the remaining portion of the list to find the position of the minimum value. Once complete, it swaps the minimum value in the unsorted portion with the first value of the unsorted portion.

3. It repeats the steps 1, 2 by moving to the next value in the list and considering it to be the new minimum.

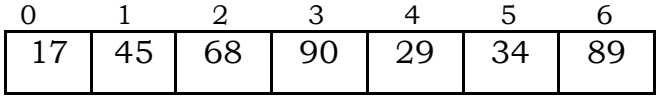
Example 1:

Consider list 89, 45, 68, 90, 29, 34, 17

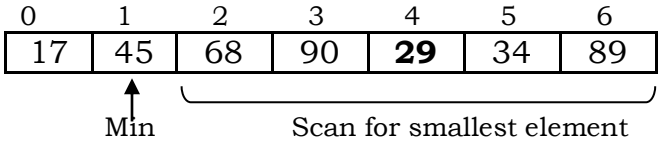
Pass 0:



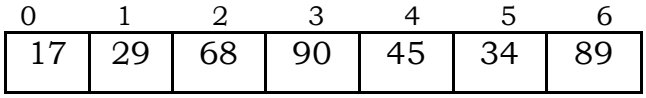
Smallest is at A[6]. Swap A[0] and A[6].



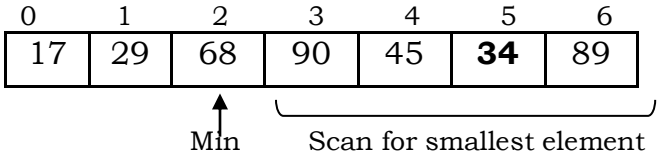
Pass 1:



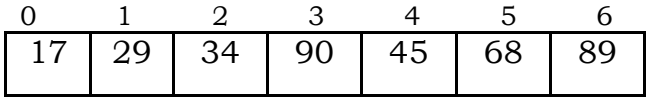
Smallest is at A[4]. Swap A[1] and A[4].



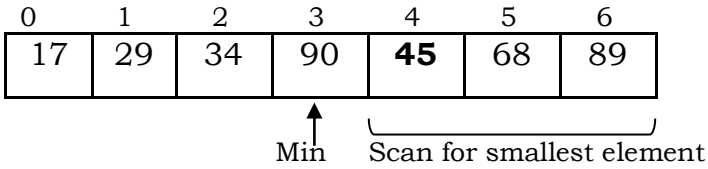
Pass 2:



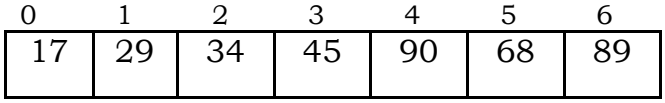
Smallest is at A[5]. Swap A[2] and A[5].



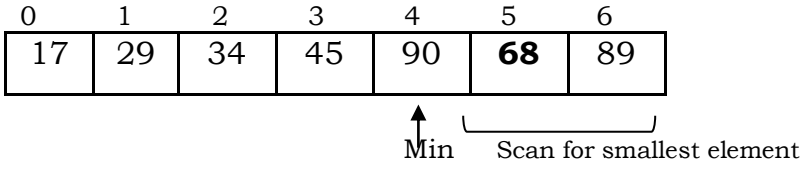
Pass 3:



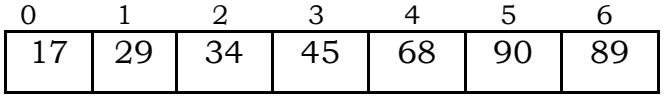
Smallest is at A[4]. Swap A[3] and A[4].



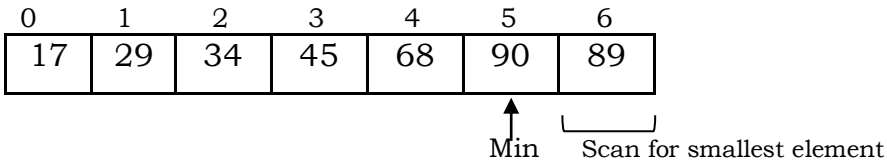
Pass 4:



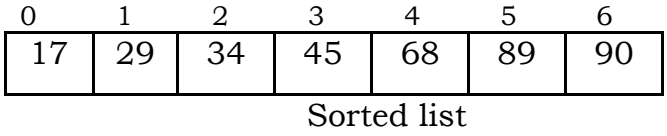
Smallest is at A[5]. Swap A[4] and A[5].



Pass 5:



Smallest is at A[6]. Swap A[5] and A[6].



Sorted list

Python code to sort a list using selection sort

```
def Selectionsort(List1):
    n =  len(List1)
    for i in range(n-1):
        min = i

        # Find the position of minimum element
        for j in range(i+1, n):
            if List1[j] < List1[min]:
                min = j

        # swap the minimum element with correct position
        if(min != i):
            List1[i],List1[min] = List1[min],List1[i]
    return List1


# Main
List1 = [65, 4, 1, 8, 11, 23, 14, 9, 45]
print("Given List = ", List1)
slist = Selectionsort(List1)
print("Sorted List  = ", slist)
```

Output:

```
Given List =  [65, 4, 1, 8, 11, 23, 14, 9, 45]
Sorted List  =  [1, 4, 8, 9, 11, 14, 23, 45, 65]
```

Time complexity of selection sort

The time complexity is used to express the number of execution steps it takes to sort the list. The implementation has two loops.

The outer loop which picks the values one by one from the list is executed n times where n is the total number of values in the list.

The inner loop, which compares the value from the outer loop with the rest of the values, is also executed n times where n is the total number of elements in the list.

Therefore,the number of comparisons is :

(n-1) + (n-2) +(n-3) + ……..+1 = n(n – 1)/2nearly equals to n^2

Complexity = $O(n^2)$

This will be the case if both loops iterate to a value that increases linearly with *n*.

Also, we can analyze the complexity by simply observing the number of loops. There are 2 loops so the complexity is **n * n = n^2**.

- **Worst case complexity**– This occurs when the input list is in **descending order**. The algorithm performs the maximum number of executions which is expressed as **$O(n^2)$**
- **Best case complexity**–This occurs when the input list is **already sorted**. The algorithm performs the minimum number of executions which is expressed as **$O(n^2)$**
- **Average case complexity**– This occurs when the input list is in **random order**. The average complexity is expressed as **$O(n^2)$**

The complexity of the selection is same in all cases. At every step, find the minimum element and put it in right place. The minimum element is not known until the end of list is not reached.

Space Complexity

The selection sort has a space complexity of O(1) as it requires one temporary variable used for swapping values.

Selection Sort	
Time Complexity	
Best case	$O(n^2)$
Worst case	$O(n^2)$
Average case	$O(n^2)$
Space Complexity	O(1)

Advantages of Selection Sort

- It performs very well on small lists
- It is an in-place algorithm. It does not require a lot of space for sorting. Only one extra space is required for holding the temporary variable.

Disadvantages of Selection Sort

- It performs poorly when working on huge lists.
- The number of iterations made during the sorting is n^2, where n is the total number of elements in the list.

Searching

Searching is a process of finding a particular element among several given elements. The search is successful if the required element is found. Otherwise, the search is unsuccessful.

Linear Search

Linear search is a sequential searching algorithm. Start from one end and check every element of the list until the desired element is found. It is the simplest searching algorithm.

Algorithm for linear search

1. Start from the leftmost element of given list and one by one compare key with each element of list
2. If key matches with any of the element, return the index value.
3. If key doesn't match with any of elements in list, return -1 or element not found.

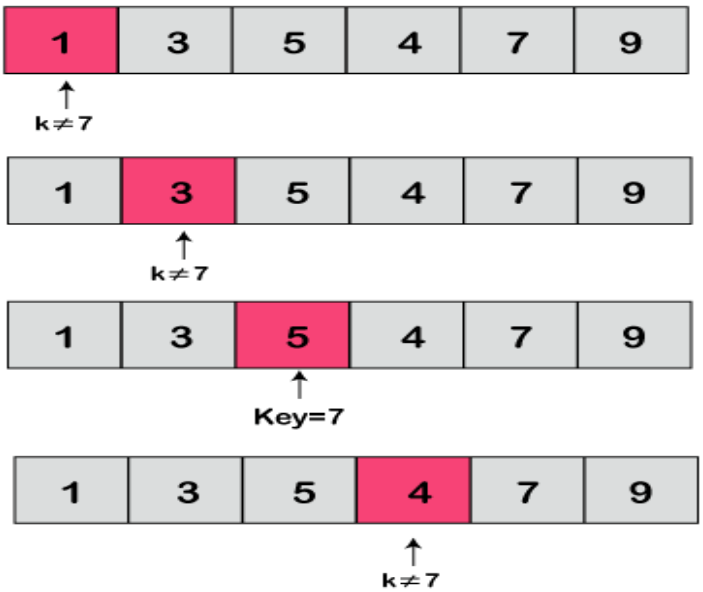
Example:Linear search to find the element **key = 7** in the given list.



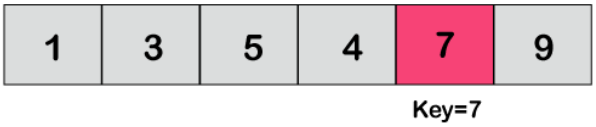
List to be Searched for

Start the search from the first

element and Compare key = 7 with each element of list .



If element is found, return the index position of the key. If element is not found, return element is not present



Python code for linear search

```
def linearsearch(list1, key):
    n = len(list1)
    # Searching list1 sequentially
    for i in range(0, n):
        if (list1[i] == key):
            return i
    return -1

# Main
list1 = [1 ,3, 5, 4, 7, 9]
key = int(input("Enter key to be searched : "))

pos = linearsearch(list1, key)

if( pos == -1):
    print("Element not found")
else:
    print("Element found at index: ", pos)
```

Output 1:

```
Enter key to be searched : 4
Element found at index:  3
```

Output 2:

```
Enter key to be searched : 12
Element not found
```

Linear Search Complexity

Time complexity of linear search algorithm –

- Base Case - O(1)
- Average Case - O(n)
- Worst Case -O(n)

Best Case Complexity

- **The element being searched is found in the first position of the list.**
- In this case, the search ends with a single comparison.
- Thus, in the best-case, the linear search algorithm performs O(1) operations.

Worst Case Complexity

- **The element being searched may be at the last position in the list or not present.**
- In the first case, the search succeeds in 'n' comparisons.
- In the next case, the search fails after 'n' comparisons.
- Thus, in the worst-case scenario, the linear search algorithm performs O(n) operations.

Average Case Complexity

The element to be searched is in the middle of the array, the average case of the Linear Search Algorithm is O(n).

Space Complexity of Linear Search Algorithm

The linear search algorithm takes up no extra space. Its space complexity is O(1) for a list of n elements.

Linear Search	
Time Complexity	
Best case	O(1)
Worst case	O(n)
Average case	O(n)
Space Complexity	O(1)

Application of Linear Search Algorithm

The linear search algorithm has the following applications:

- Linear search can be applied to both single-dimensional and multi-dimensional arrays.
- Linear search is easy to implement and effective when the array contains only a few elements.
- Linear Search is also efficient when the search is performed to fetch a single search in an unordered-List.

Decrease-and-Conquer technique

This approach is also known as incremental or inductive approach.

The decrease-and-conquer design technique include following steps:

- **Decrease** or reduce problem instance to smaller instance of the same problem.
- **Conquer** the problem by solving smaller instance of the problem
- **Extend** solution of smaller instance to obtain solution to original problem

Example of decrease and conquer design technique – Insertion sort

Insertion sort

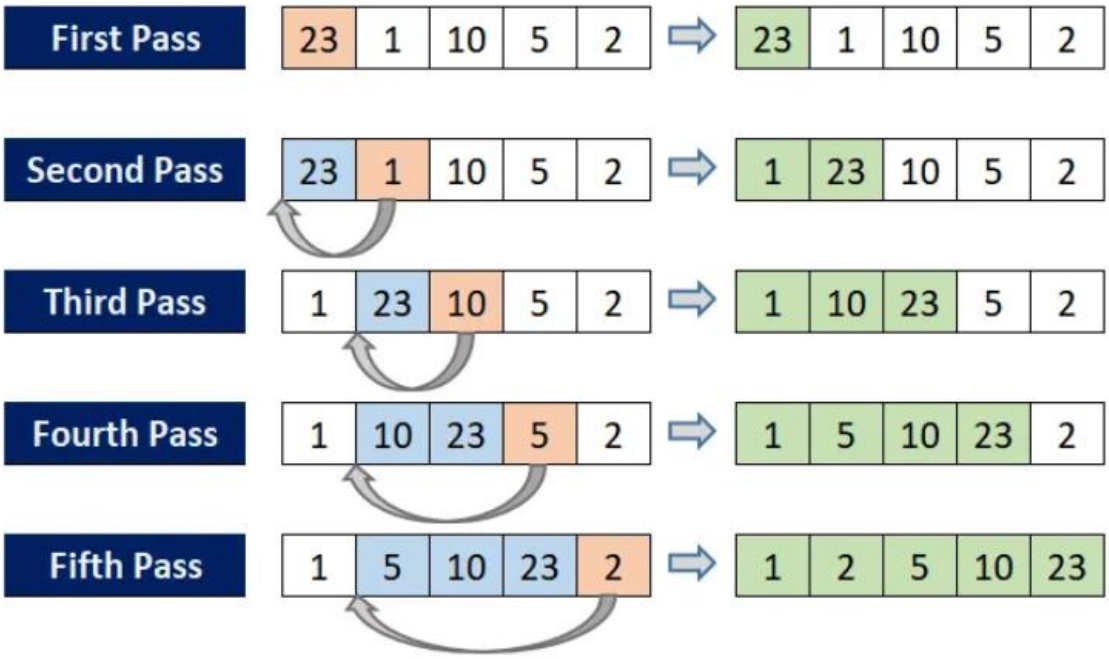
Insertion sort is based on the idea of taking one element from unsorted list and inserting it at the correct position in the sorted list. This will result into increasing the length of the sorted array by one and decreasing the length of unsorted array by one after each iteration.

In the insertion sort algorithm, it is assumed that a portion of the list has already been sorted, while the other portion is unsorted. Picking one element at a time from unsorted portion of list and find appropriate position among the sorted portion of list and insert it there. This is done by scanning the sorted portion of the list from right to left until the first element smaller than or equal to it is found. Insert the element after that.

Algorithm of Insertion sort -

1. Spilt a list in two parts - sorted and unsorted.
2. Traverse the given list from second element to end of the list.(list[1] to list[n])
3. Picking one element at a time from unsorted portion of list and find appropriate position among the sorted portion of list and insert it there. This is done by scanning the sorted portion of the list from right to left until the first element smaller than or equal to it is found. Insert the element after that.
4. Repeat the steps 3, until sorted list is obtained. At each iteration sorted list grows.

Example:



Python code for Insertion sort

```
def insertionsort(num):
    n = len(num)
    for i in range (n-1):
        ele = num[i]
        j = i - 1

        #Compare ele with each element in sorted list,
        #until an element smaller than it is found

        while j >= 0 and ele < num[j]:
            num[j+1] = num[j]
            j = j - 1

        #Place ele  after the element just smaller than it
        num[j+1] = ele
    return num


# Main
list1 = [45, -12, 64, 9, 5, 15]
print("Given list = ", list1)
slist = insertionsort(list1)
print("Sorted array :", slist)
```

Output:

```
Given list =  [45, -12, 64, 9, 5, 15]
Sorted array : [-12, 5, 9, 45, 64, 15]
```

Insertion Sort Time Complexity

- **Best case complexity O(n)**

List is already **sorted**, the outer loop runs for n number of times whereas the inner loop does not run at all. So, there are only n number of comparisons. Thus, complexity is linear.

- **Worst case complexity $O(n^2)$**

List is in **descending order**, and want to sort it in ascending order. In this case, worst case complexity occurs. Each element has to be compared with each of the other elements so, for every n^{th}element, (n-1) number of comparisons are made. Thus, the total number of comparisons = n * (n-1) ~ n^2

- **Average case complexity $O(n^2)$**

It occurs when the elements of a list in random order (neither ascending nor descending).

Space Complexity

Space complexity is O(1) because an extra variable is used.

Insertion Sort	
Time Complexity	
Best case	O(n)
Worst case	$O(n^2)$
Average case	$O(n^2)$
Space Complexity	O(1)

Week – 4 Divide and Conquer

Divide and conquer

Divide-and-conquer is the best-known algorithm design technique for problem solving.

Divide-and-conquer algorithms work as follows:

1. A problem is divided into several sub problems of the same type, ideally of about equal size.
2. The sub problems are solved (typically recursively).
3. If necessary, the solutions to the sub problems are combined to get a solution to the original problem.

Examples for divide and conquer technique are – Merge sort, quick sort, Binary search

Merge Sort

Merge sort works on the concept of divide and conquer. It is one of the most popular and efficient sorting algorithms.

Merge sort works as follows:Divide the unsorted list into sub lists again and again into halves until list with one element each is obtained. Then combine the pair of one element lists into two element lists, sorting them in the process. The sorted two element pairs are merged into the four element lists, and so on until the sorted list is obtained.

The ***merging*** of two sorted lists is done as follows.

1. Two variables are initialized to point to the first elements of the lists being merged.
2. The elements pointed to are compared, and the smaller of them is added to a new list being constructed. After that, the index of the smaller element is incremented to point to its immediate successor in the list it was copied from.
3. This operation is repeated until one of the two given lists is exhausted, and then the remaining elements of the other list are copied to the end of the new list.

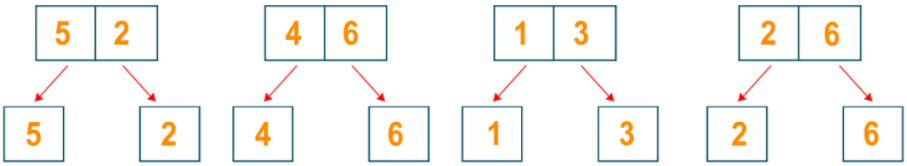
Example 1: Consider unsorted list of elements as shown below



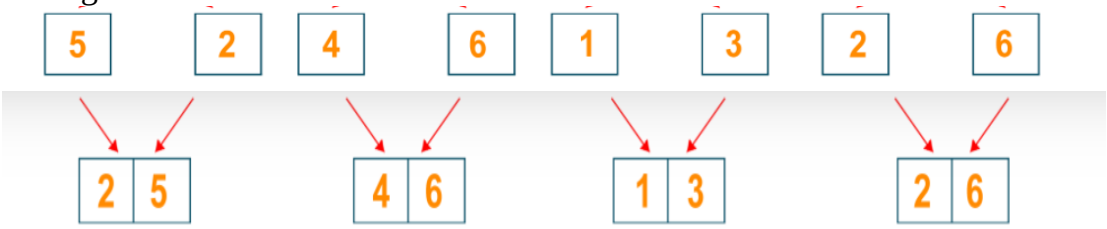
Divide it into two halves as given below



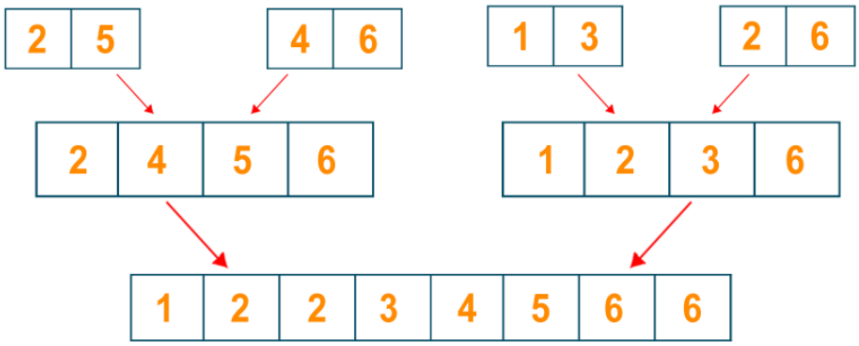
Keep dividing it until the subpart of the list consists of only one element



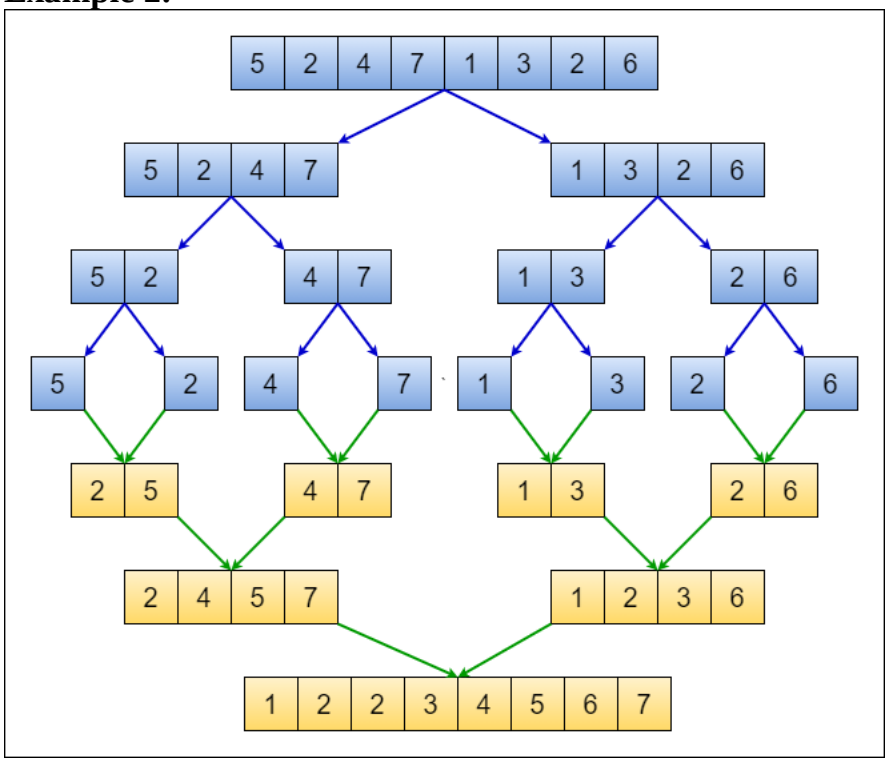
Combine the two list of one element and convert into a list of two element after sorting.



Again, combine the two lists of 2 elements and convert into a list of 4 elements after sorting. Keep repeating the process until sorted list is obtained.



Example 2:



Time and Space Complexity of Merge Sort

The time complexity of merge sort is *O(nlogn)*. This is because the list is split in *log(n)* calls and the merging process takes linear time in each call.

The time complexity for **best case, worst case, and average-case** is **-O(nlog n)** where n is the number of elements to be sorted.

The space complexity of the algorithm is **O(n)** where alist(array) of size n is needed to place the sorted element.

Time Complexity	
Bestcase	O(n log n)
Worst case	O(n log n)
Average case	O(n log n)
Space Complexity	O(n)

Advantages of Merge Sort

- Merge sort allows sorting large data set easily
- Merge sort can access data in sequence, hence the need for random access is low
- Merge sort is a stable sorting algorithm
- It has the same running time complexity for best, worst and average case.

Disadvantages of Merge Sort

- It requires an array of the same size as the original list to store the final sorted array which consumes the large memory space
- It is slower when it comes to sorting smaller data sets

Applications

- Merge sort is used in e-commerce application
- Merge sort is the best way to sort the linked list in comparison to another sorting algorithm
- Merge sort is used in internal and external sorting
- It is used in organizing MP3 library
- It is used to display Google PageRank results
- Merge sort is widely used in inversion count problem

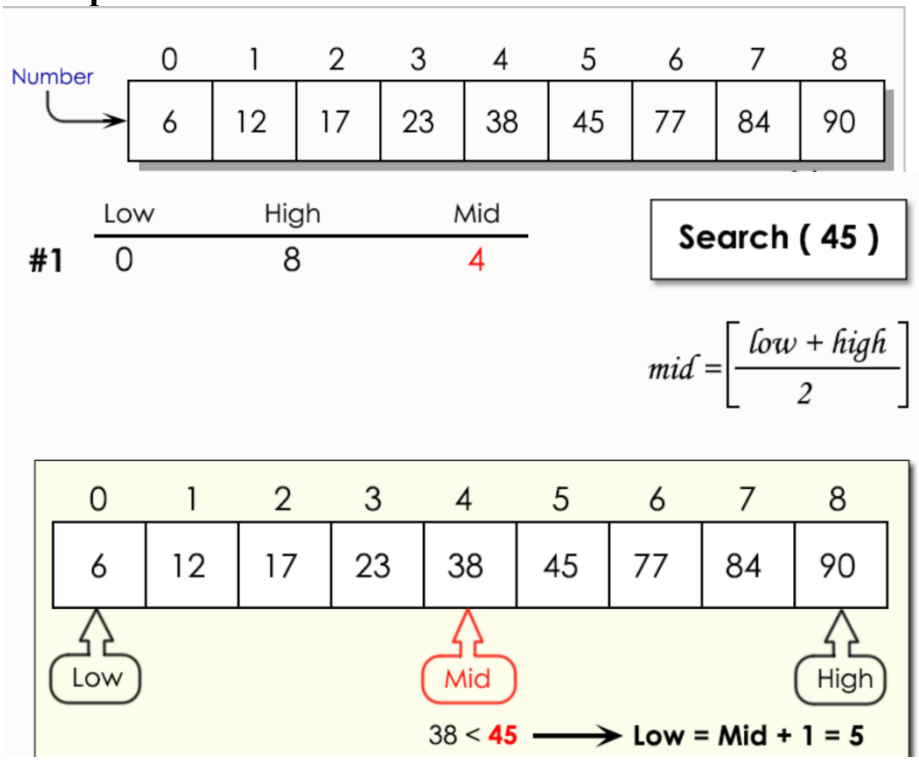
Binary Search

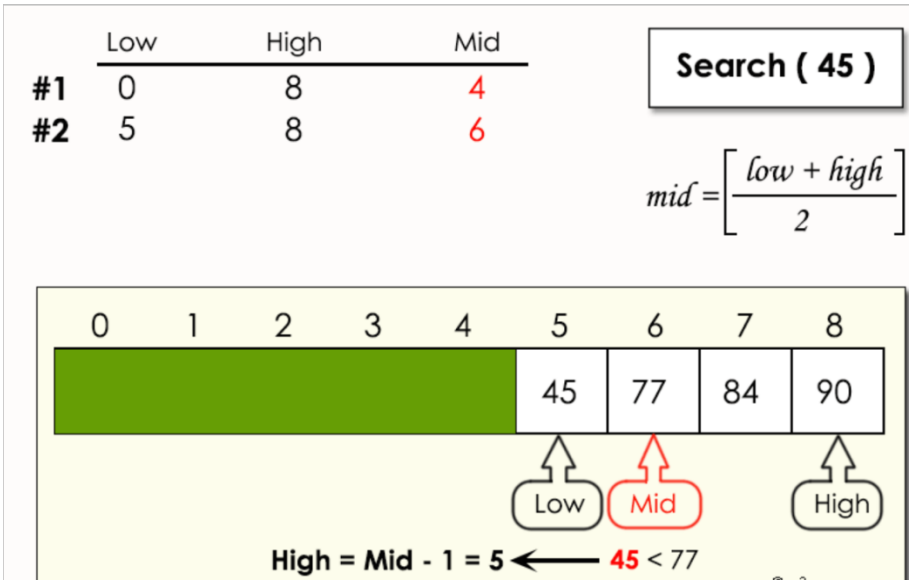
Binary search is a searching algorithm which is used to search an element from a **sorted**list. Binary search is an efficient algorithm and is better than linear search in terms of time complexity.Binary Search uses divide and conquer technique

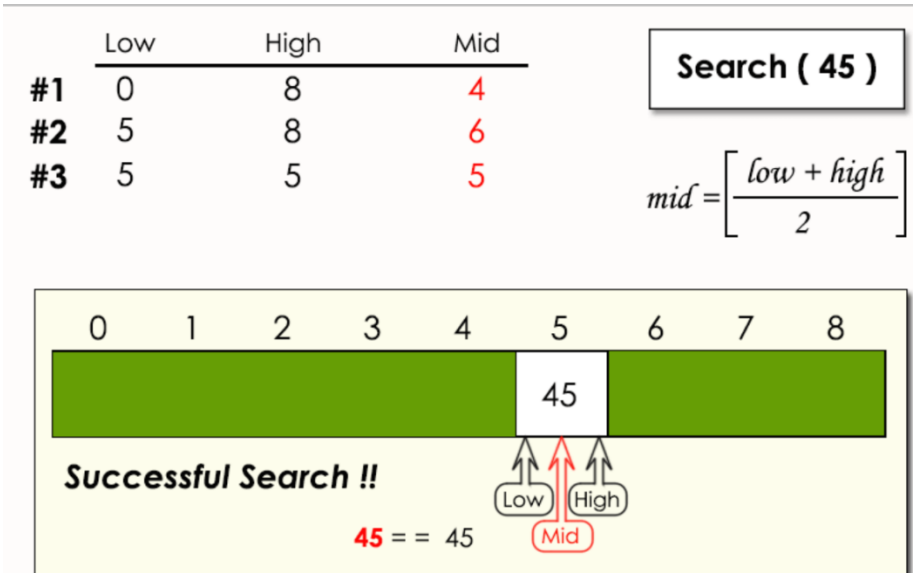
Algorithm of binary search

1. It compares a search key *with* the list's middle element. If they match, then index of matching element is returned and stops.
2. If search key is lesser than middle element, then key is searched in sub list to the left of middle element.
3. If search key is greater than middle element then key is searched in the sub list to the right of the middle element.
4. This process continues on the sub list as well until the size of the sublist reduces to zero.

Example:







Time Complexity of Binary Search

The running time complexity for binary search is different for different case of input –

- **Best case ComplexityO(1)**

The best-case time complexity is O(1) which means the element is located at the middle of the list and the element is found in first comparison.

- **Average and Worst-Case time complexity O(log n)**

The average and worst case complexity is O(log n) which means the element to be found is located either on the left side or on the right side of the middle. Here, n indicates the number of elements in the list.

Binary search is efficient and faster-searching algorithm but can be used only for searching from a sorted array.

Space Complexity
The space complexity of the binary search algorithm is O(1).

Binary search can be implemented using the following methods

- Recursive Method
- Iterative Method

Python code for Binary Search (Recursive Method)

```
def BinarySearch(mylist, key, low, high):
    if low <= high :

        mid = (low + high) // 2

        if key == mylist[mid] :
            return mid

        elif key < mylist[mid] :
            return BinarySearch(mylist, key, low, mid-1)

        else:
            return BinarySearch(mylist, key, mid + 1, high)

    else:
        return -1

#Main
mylist = [11, 12,13, 14, 15, 16, 17, 18]
key = int(input("Enter key  "))
n = len(mylist)
pos = BinarySearch(mylist,key, 0, n-1)
if pos == -1:
    print("UnSuccessfull search")
else:
    print("Sucessfull search: Element found at : ",pos)
```

Output:

```
Enter key  13
Sucessfull search: Element found at :  2
```

Difference between Linear search and Binary Search

Sl No	Binary Search	Linear Search
1	Search the position of the searched element by finding the middle element of the array	Starts searching from the first element and compares each element with a search key
2	Need the sorted list of elements	Do not need the sorted list of elements
3	Cannot be directly implemented on linked-list	Can be implemented on array and linked-list
4	Divide and Conquer in nature	Iterative in nature
5	Preferred for a large size data set	Preferred for a small size data set

Dynamic Programming

Dynamic Programming is an algorithmic design technique for solving a problem by breaking it into similar subproblems and saves the result of these subproblems, so that we do not have to re-compute them when it is needed.

Recursion is the process in which a function calls itself until the base cases are reached. And during the process, complex situations will be traced recursively and become simpler and simpler. The whole structure of the process is tree like. Recursion **does not store any value** until reach to the final stage (base case).

A **recursive** solves a particular problem by calling itself and solving smaller subproblems of the original problems.In recursion, also break a problem into similar subproblems. The difference is, in recursion similar subproblems may be solved multiple times. But in dynamic programming, we keep track of already solved subproblems and don't solve them more than once.
The problems that appear multiple times are called**overlapping subproblems**. To solve a problem in dynamic programming, save the solutions of overlapping subproblems in a data structure. This is known as "**Memoization**".
To solve a problem using dynamic programming, the problem must has the following attributes:

- **Optimal subproblems:** Able to break a larger problem into similar subproblems. If subproblems can be solved optimally, can use them to solve the main problem.
- **Overlapping subproblems:** The recursive solution to the problem contains subproblems that are repeated multiple times.

Example: Fibonacci Series

Fibonacci number series is0, 1, 1, 2, 3, 5, 8, 13, 21, 34 . . .

Every number in the series is the sum of the previous two numbers except the first two numbers. The first number is 0 and the second number is 1. They are fixed. The third number is 1 which is the sum of the previous two numbers 0 and 1. The fourth number is 2 which is the sum of the previous two numbers 1 and 1. And the series goes on like this.

Consider function fib(n) that returns the nth number of the Fibonacci series.

To solve a Fibonacci series problem recursively, break the problem into similar subproblems and base cases are needed.

For Fibonacci series the **base cases** are –

fib (0) = 0

fib (1) = 1

Break the problem into similar subproblems with the help of formula —

fib(n) = fib(n-1) + fib(n-2) **if n > 1**

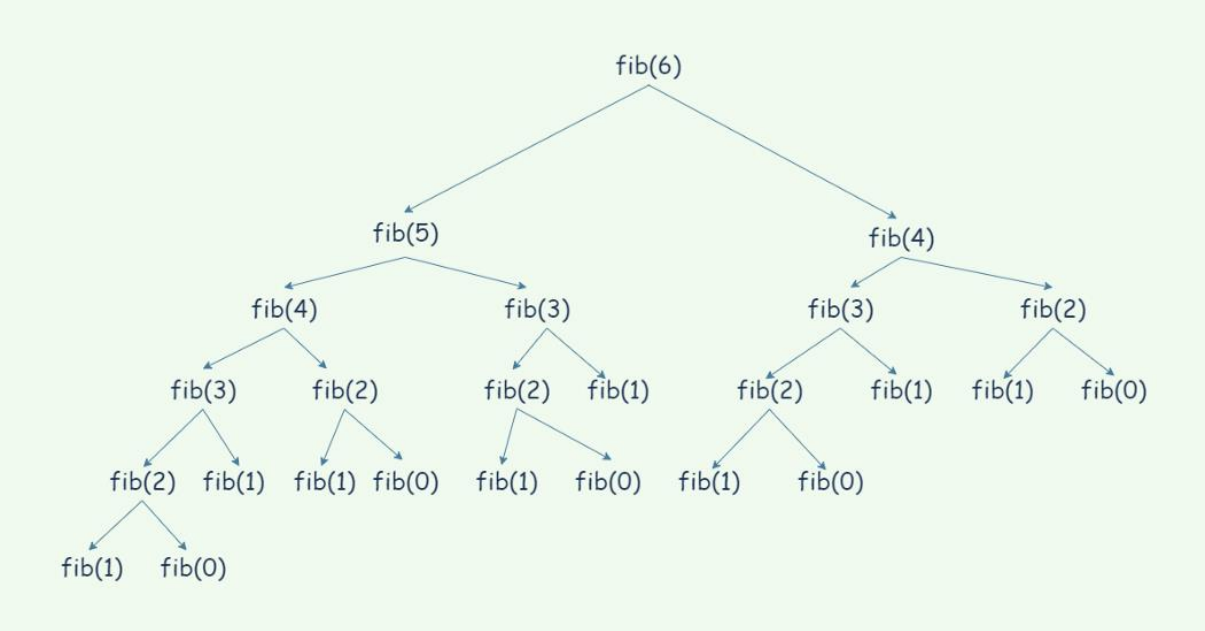
Python code for Fibonacci series using recursion is

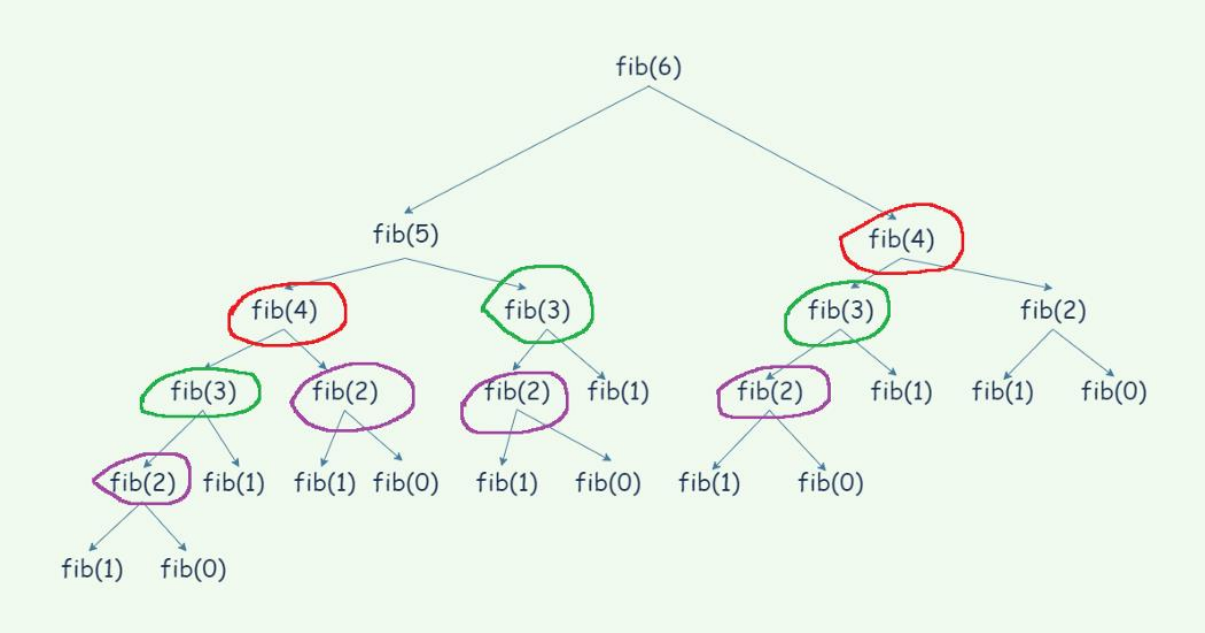
```
def fib(n):
    if n == 0 :
        return 0
    if n == 1 :
        return 1
    return  fib(n-1) + fib(n-2)

# Main
print(fib(6))

Output:
    8
```

Let's analyse the recursion tree for **fib(6).** When the function **fib(6)** is called the recursion operates like this —





Overlapping subproblems are –

fib(4) is being called 2 times, **fib(3)** is being called 3 times and **fib(2)** is being called 4 times. Ignore the base cases.

In dynamic programming, the overlapping subproblems are solved only once.

Dynamic Programming Approach for Fibonacci Series

- Store the results of previously solved subproblems in a data structure like a list.

- Function fib() will check if a subproblem is already solved or not. If solved before,not solve it again.
- create list memo[]. Once a subproblem is solved store the solution in memo[]. Saving the solutions like this is called **memoization**.
- Populate the list memo[] with a value that can never be a solution. In our case -1
- Now for fib(n) check if memo[n] is equal to -1 or not. If it is -1 then problem was not solved before and will solve it. But if it is other than -1 then problem was solved before. Sodon't have to solve it again. Just return it.

Python code -Dynamic Programming for Fibonacci Series

```
def fib(n):
    if n == 0:
        return 0
    if n == 1:
        return 1

    if memo[n] != -1:
        return memo[n]

    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]


#Main
memo = []
for i in range(10):
    memo.append(-1)

print("Fibonacci number is :",fib(6))
```

Output:

```
Fibonacci number is : 8
```

Comparison of time complexity of Recursive approach and Dynamic programming approach

In the **recursive approach** for every value from 0 to n two functions are called (except the base cases). For example, fib(6) is calling fib(5) and fib(4). Then fib(5) is calling fib(4) and fib(3). Again fib(4) is calling fib(3) and fib(2) and so on. So the time complexity of the recursive approach is O(2^n). Which is an **exponential complexity**. A code with exponential complexity is very slow and not efficient at all.

space complexity is O(N) since the depth of the tree will be proportional to the size of n.

In the **dynamic programming** approach, subproblem is solved once. For fib(n) there are total n subproblems — fib(0), fib(1), fib(2) … fib(n). And each one of them is solved once.So, the time complexity for the dynamic approach is **O(n).** It is **a linear complexity**. So solving a problem using dynamic programming have a huge improvement in performance.

Space complexity is O(N) since all intermediate values are stored.

And **Dynamic Programming** is mainly an optimization compared to simple recursion. In terms of big O, this optimization method generally reduces time complexities from exponential to polynomial.

Features\Algorithm	Recursion	Dynamic Programming
Coding Logic	Base Case + Pattern	Base Case + Pattern
Store Intermediate Result?	No	Yes
Procedure Plot	Tree Like	Linear
O (Time Complexity)	O(2^n)	O(n)
O (Space Complexity)	O(n)	O(n)

Backtracking

A backtracking algorithm is a problem-solving algorithm that uses a brute force approach for finding the desired output.

The Brute force approach tries out all the possible solutions and chooses the desired /best solution

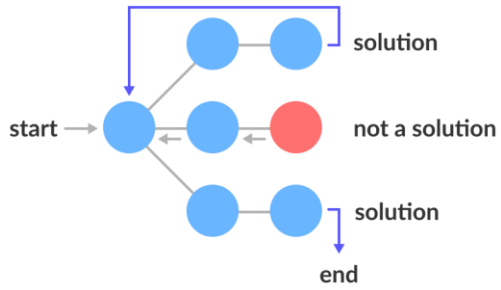
The term backtracking suggests that if the current solution is not suitable, then backtrack and try other solutions.

Backtracking is an algorithmic technique for solving problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point in time.

This approach is used to solve problems that have multiple solutions

There are three types of problems in backtracking –

1. Decision Problem – In this, search for a feasible solution.
2. Optimization Problem – In this, search for the best solution.
3. Enumeration Problem – In this, find all feasible solutions.



Example for backtracking are: Eight queen problem, soduku

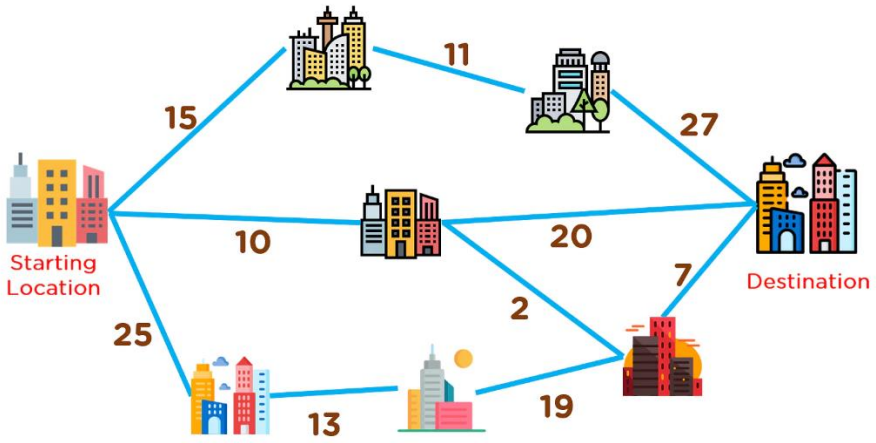
Greedy Technique

The greedy approach suggests constructing a solution through a sequence of steps, each expanding a partially constructed solution obtained so far, until a complete solution to the problem is reached.

On each step the choice made must be:

1. ***Feasible,*** i.e., it has to satisfy the problem's constraints
2. ***Locally optimal,*** i.e., it has to be the best local choice among all feasible choices available on that step

3. ***Irrevocable,*** i.e., once made, it cannot be changed on subsequent steps of the algorithm



Example for greedy technique are -

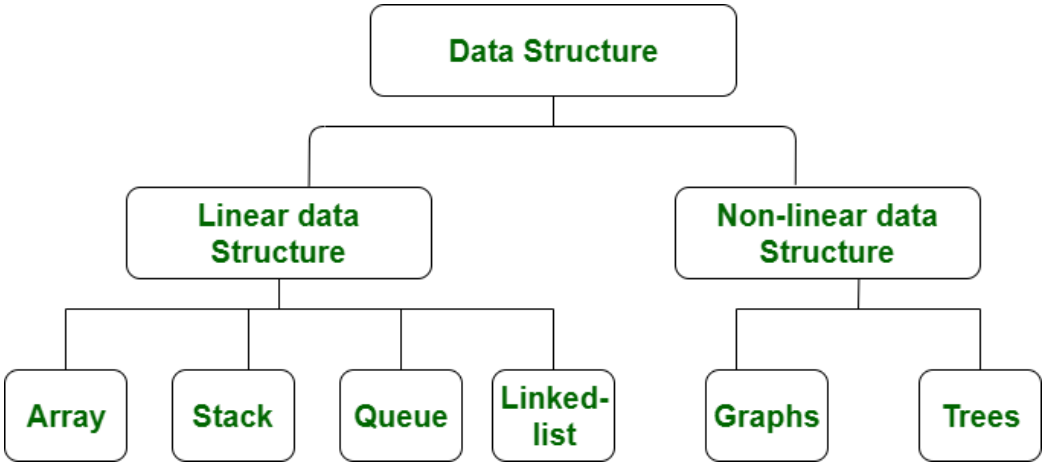
Kruskal's algorithm and **Prim's algorithm** for finding minimum spanning trees.

Dijkstra's algorithm to find shortest route from source to all other nodes.

~~~~~~
~~~~~~

WEEK – 5 LINKED LIST

In **linear data structure**, data elements are sequentially connected and each element is traversable through a single run. In **non-linear data** structure, data elements are hierarchically connected and are present at various levels. In linear data structure, all data elements are present at a single level.



Array - An array is defined as a collection of items of same type stored in contiguous memory location. It is used to store multiple values in single variable. Array holds fixed number of elements.

Array is created in Python by importing array module to the python program. Array is declared as shown below

```
from array import *
arrayname = array ( typecode, [Intilaizers] )
```

Typecode are the codes that are used to define the type of value the array will hold.

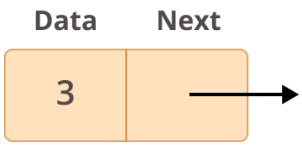
```
from array import *
# Creating array of signed integers
a1 = array('i' , [5, 3, 4, 7, 9])
```

Stack: It is linear data structure that uses the LIFO (Last In-First Out) i.e., data added last will be removed first. The addition of data element in a stack is known as a **push** operation, and the deletion of data element form the list is known as **pop** operation.

Queue: It is a data structure that uses the FIFO (First In-First Out) i.e., the element which is added first will be removed first. There are two terms used in the queue **front** end and **rear end.** The insertion operation performed at the back end is

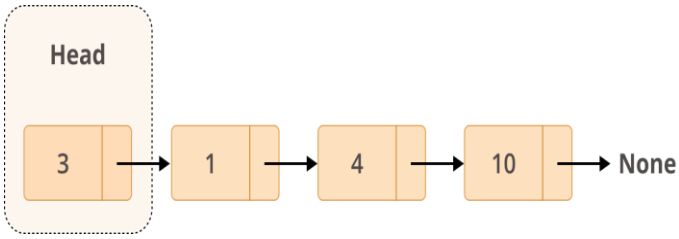
known as enqueue, and the deletion operation performed at the front end is known as dequeue.

Linked list: It is a collection of nodes that are made up of two parts, i.e., data element and reference to the next node in the sequence.



1. **Data** contains the value to be stored in the node.
2. **Next** contains a reference to the next node on the list.

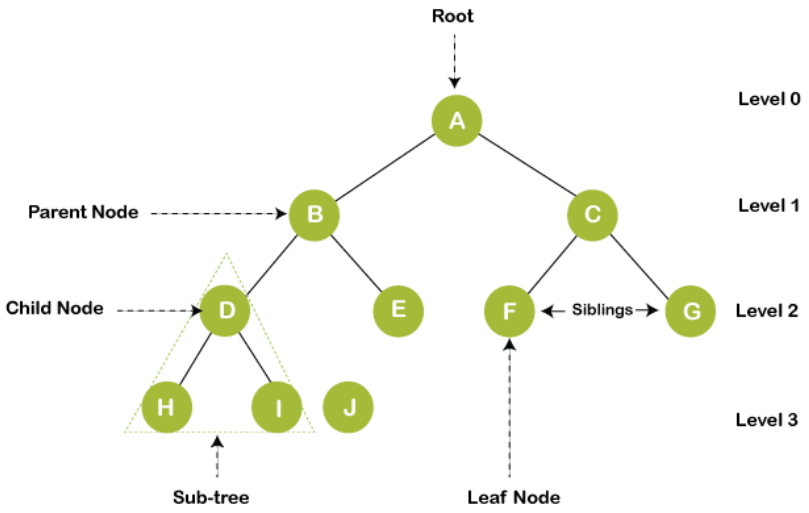
A linked list is a collection of nodes. The first node is called the **head**, and it's used as the starting point for any iteration through the list. The last node must have its next reference pointing to None to determine the end of the list. Figure shows linked list



A **non-linear data structure** is also another type of data structure in which the data elements are not arranged in a contiguous manner. As the arrangement is nonsequential, so the data elements cannot be traversed or accessed in a single run. In the case of linear data structure, element is connected to two elements (previous and the next element), whereas, in the non-linear data structure, an element can be connected to more than two elements.
Trees and **Graphs** are the types of non-linear data structure.

Tree : It is a non-linear data structure that consists of various linked nodes. It has a hierarchical tree structure that forms a parent-child relationship. The diagrammatic representation of a **tree** data structure is shown below:

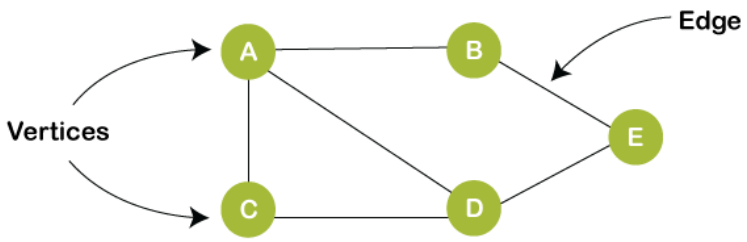
For example, the posts of employees are arranged in a tree data structure like managers, officers, clerk.



Graph : A graph is a non-linear data structure that has a finite number of vertices and edges, and these edges are used to connect the vertices. The vertices are used to store the data elements, while the edges represent the relationship between the vertices.

Agraph is used in various real-world problems like telephone networks, circuit networks, social networks like LinkedIn, Facebook. In the case of Facebook, a single user can be considered as a node, and the connection of a user with others is known as edges.

The benefit of linked list over an array or list is that elements can be easily inserted and removed without the need of changing the index of all other items. The memory used to store the linked list does not need to be reorganised because the data does not have to be stored contiguously. However, can't access items in constant time (O(1)) as it is possible in an array. The time complexity for looking up an item in the linked list has a linear time complexity (O(n)).



The following are the differences between linear and non-linear data structures –

Sl. No	Key	Linear Data Structure	Non-Linear Data Structure
1	Data Element Arrangement	In linear data structure, data elements are sequentially connected.	In non-linear data structure, data elements are hierarchically connected.
2	Levels	In linear data structure, all data elements are present at a single level.	In non-linear data structure, data elements are present at multiple levels.
3	Implementation complexity	Linear data structures are easier to implement.	Non-linear data structures are difficult to understand and implement as compared to linear data structures.
4	Traversal	Linear data structures can be traversed completely in a single run.	Non-linear data structures are not easy to traverse and needs multiple runs to be traversed completely.
5	Memory utilization	Linear data structures are not very memory friendly and are not utilizing memory efficiently.	Non-linear data structures uses memory very efficiently.
6	Time Complexity	Time complexity of linear data structure often increases with increase in size.	Time complexity of non-linear data structure often remain with increase in size.
7	**Applications**	Linear data structures are mainly used for developing the software.	Non-linear data structures are used in **image processing** and **Artificial Intelligence**.
8	Examples	Array, List, Queue, Stack.	Graph, Map, Tree.

Linked List

A linked list is a linear data structure. It doesn't store the data in contiguous memory locations like arrays. And each element in linked list is called a **node** and they are connected using the **pointers**. The first **node** in the linked list is called the **head**.

The size of the linked list is dynamic. So, can have any number of nodes as we want unless the storage is available in the device.

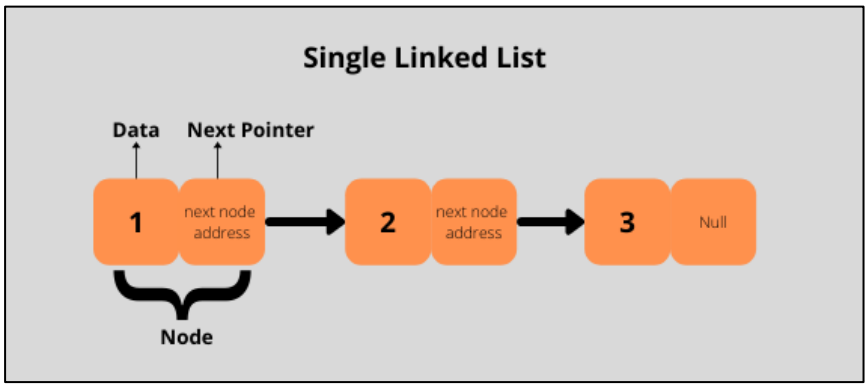
There are two types of linked lists. They are –

1. Singly Linked List
2. Doubly Linked List

Singly Linked List

A singly linked list contains a **single pointer** connected to the **next node** in the linked list. We have to store the data and pointer for each node in the linked list.

The last node in the linked list contains **null** as the next pointer to represent the ending of the linked list. It can only be traversed in a single direction i.e., can go from first node in a list to last node but cannot move from last node to first node.



Singly Linked List Implementation

Creating the node of linked list –

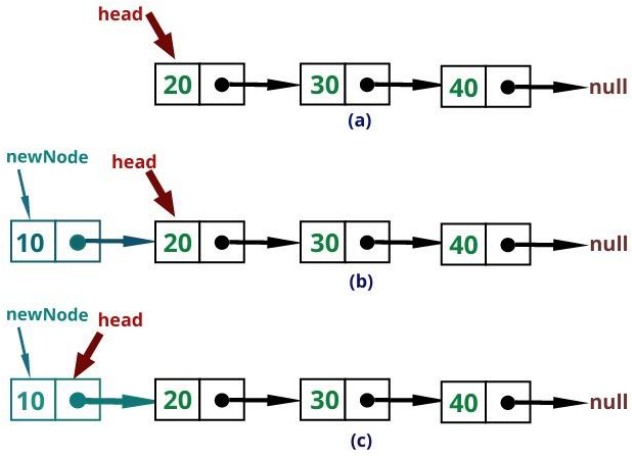
1. Create the **node** using the **class**. The class contains **data** and a **pointer** for the **next node**. Create the node with any type of data.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

2. Create a class called **LinkedList** with head initialized to **None**.

```
class Linkedlist:
    def __init__(self):
        self.head = None
```

3. **Inserting a new node at the beginning of the linked list**-
   - Create a new node with the given data.
   - Make the new node points to the head node.
   - Make the new node as the head node.




```
class Linkedlist:
    def __init__(self):
        self.head = None

    def addbegining(self,data):
        newnode = Node(data)
        newnode.next = self.head
        self.head = newnode
```

4. **To print all nodes data in the linked list**-

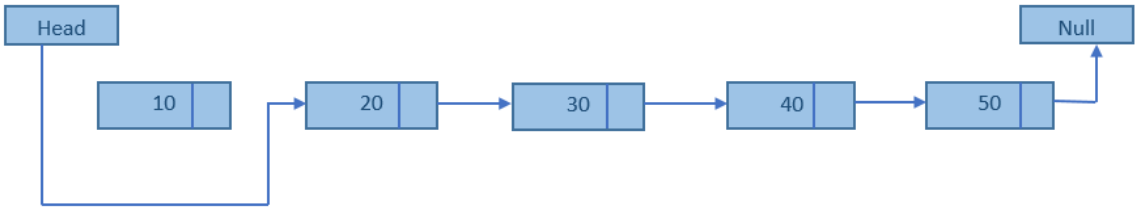
- Initialize a variable with **head**.
- Loop that iterates until the end of the linked list is reached.
  - Print the node data.
  - Move the next pointer

```
def display(self):
    temp = self.head
    while temp != None:
        print(temp.data, end=' -> ')
        temp = temp.next
    print('Null')
```

5. Deletion at the beginning of linked list

- Point head to the second node

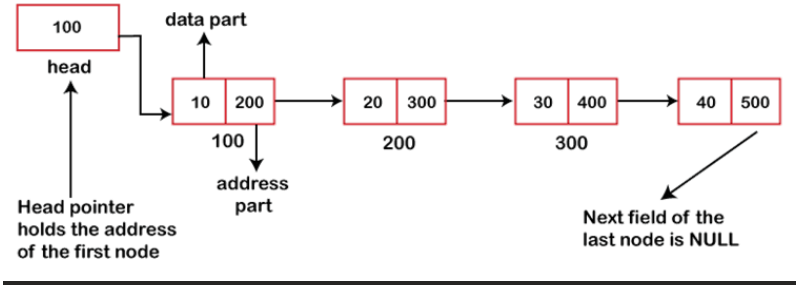
```
def deletebegining(self):
    self.head = self.head.next
```



6. Search a node by value in linked list

- Make head as the current node.
- Run a loop until the current node is NULL.
- In each iteration, check if the key of the node is equal to item. If the key matches the item, return True otherwise return False.

```
def search(self, key):
    temp = self.head
    while temp != None:
        if temp.data == key :
            return True
        else:
            temp = temp.next
    return False
```



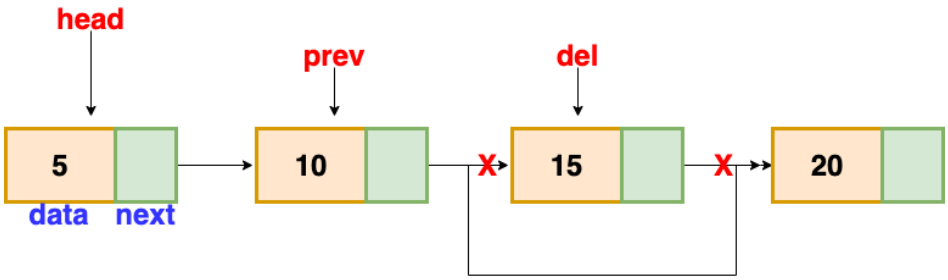
7. Delete a node by value

To delete a node from the linked list, we need to do the following steps.

1) Find the previous node of the node to be deleted.

2)Change the next of the previous node.

3) Free memory for the node to be deleted.



Deleting a Node in Linked List

```
    def deleteNode(self, key):
        # Store head node
        temp = self.head

        # If head node itself holds the key to be deleted
        if (temp is not None):
            if (temp.data == key):
                self.head = temp.next
                temp = None
                return

        # Search for the key to be deleted, keep track of the
        # previous node as we need to change 'prev.next'
        while(temp is not None):
            if temp.data == key:
                break
            prev = temp
            temp = temp.next

        # if key was not present in linked list
        if(temp == None):
            return
```

```
    # Unlink the node from linked list
    prev.next = temp.next
    temp = None
```

Time Complexity of Linked List

	Worst case	Average Case
Search	O(n)	O(n)
Insert	O(1)	O(1)
Deletion	O(1)	O(1)

Advantage of Linked List

- Dynamic memory allocation – memory is allocated and deallocated efficiently.
- Easy implementation

Disadvantage of Linked List

- Certain memory is used by pointers.
- Traversal is time consuming as direct access is not possible.

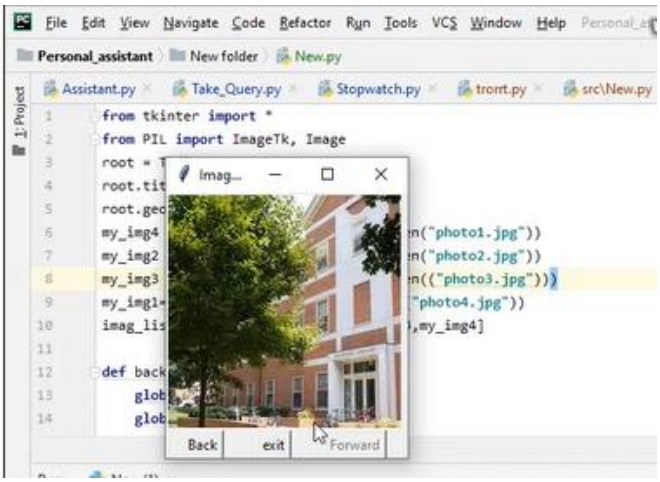
Applications

- Linked list is used in –
- Implementation of stacks and queues
- Implementation of graph
- Performing arithmetic operations on long integers.

Examples

Image viewer

An image is a program that can display stored graphical images. It can handle various file formats. An image viewer links all the images with each other. Images are viewed by clicking the next and previous buttons easily as each node in a linked list has a pointer to the previous and next node respectively. It is easy to delete an image in the middle of list.



Music Player list

In music player, each song is linked to previous and next songs. Can play songs from starting or ending of the list.

The functions like – skip back/ forward, play next, add can be added using linked list.

Python Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Linkedlist:
    def __init__(self):
        self.head = None

    def AddNode(self,data):
        newnode = Node(data)
        newnode.next = self.head
        self.head = newnode
```

```
    def DeleteNode(self):
        # Store head node
        temp = self.head

        # If head node is empty
        if (temp != None):
            self.head = temp.next
            temp = None
            return

    def Search(self, key):
        temp = self.head
        while temp != None:
            if temp.data == key :
                return True
            else:
                temp = temp.next
        return False
```

```
    def Display(self):
        temp = self.head
        while temp != None:
            print(temp.data, end='->')
            temp = temp.next
        print('Null')

#Main
L1 = Linkedlist()
L1.AddNode(5)
L1.AddNode(15)
L1.AddNode(35)
L1.AddNode(45)
print("Linked List after adding :")
L1.Display()
L1.DeleteNode()
print("Linked List after deleting :")
L1.Display()
res = L1.Search(10)
if res == True:
    print("Searching : Element present")
else:
    print("Searching : Element Not present")
```

Output:

```
Linked List after adding :
45->35->15->5->Null
Linked List after deleting :
35->15->5->Null
Searching : Element Not present
```